

AULA 12 • PROGRAMAÇÃO ORIENTADA A OBJETOS

JDBC e DAO

`Connection` · `PreparedStatement` · `ResultSet` · Padrão DAO —
como salvar os dados num banco de verdade, sem perder nada.

Java • P00 • **Prof. Rob Silva**

O caminho de hoje.

- 01 Por que usar um banco
- 02 JDBC: Connection, PreparedStatement, ResultSet
- 03 Conectar ao PostgreSQL
- 04 `try-with-resources`
- 05 Padrão DAO
- 06 Melhores práticas de queries

Fechou o programa? Perdeu tudo.

Até agora os produtos viviam num `ArrayList`, na memória. Isso funciona enquanto o programa está aberto — mas ao fechar, tudo some. Hoje conectamos a um banco PostgreSQL de verdade: os dados ficam salvos, mesmo depois de desligar.

Produtos num `ArrayList`. Fechou o programa, a lista some.

Produtos numa tabela do PostgreSQL. Desligou? Continua lá.

O que é JDBC.

JDBC é a forma padrão do Java conversar com bancos SQL. São 4 peças que trabalham juntas.



O driver do PostgreSQL.

A API JDBC (`java.sql`) já vem no Java — mas é genérica. Para falar com o PostgreSQL, o projeto precisa do **driver do PostgreSQL**: um arquivo `.jar` (ex.: `postgresql-42.7.x.jar`). Sem ele, o `DriverManager` não sabe conectar.

```
SQLException: No suitable driver found  
for jdbc:postgresql://localhost:5432/estoque
```

Baixe em `jdbc.postgresql.org/download` — é um único `.jar` .

Funciona no **JDK 8** e versões acima; o JDBC não é preso ao JDK 8. Só combine a versão do driver (série 42.x) com o seu Java.

No JDBC 4+ (Java 6 em diante), o driver se registra sozinho — `Class.forName("org.postgresql.Driver")` não é obrigatório (você pode ver isso em códigos antigos).

Adicionar o driver no NetBeans 8.0.2.

- 1 Baixe o `postgresql-42.7.x.jar` e salve numa pasta do projeto (ex.: `lib/`).
- 2 Na aba **Projects**, abra o seu projeto e localize o nó **Libraries**.
- 3 Clique com o botão direito em **Libraries** → **Add JAR/Folder...**
- 4 Selecione o `postgresql-42.7.x.jar` → **Open**.
- 5 O `.jar` aparece sob **Libraries** — o driver está no classpath. Agora o `DriverManager.getConnection(...)` funciona.

Projeto Maven? Em vez disso, adicione a dependência `org.postgresql:postgresql` no `pom.xml`.

Conectar ao PostgreSQL.

```
String url =  
    "jdbc:postgresql://localhost:5432/estoque";  
String user = "postgres";  
String senha = "1234";  
  
Connection con =  
    DriverManager.getConnection(url, user, senha);  
System.out.println("Conectado ao banco!");  
con.close();
```

O `DriverManager` usa a URL, o usuário e a senha para abrir a `Connection`. É a linha aberta com o banco.

Statement × PreparedStatement.

Statement

Monta a query concatenando texto — fica aberto a **SQL injection**.

```
// texto colado direto na query
String sql = "... WHERE nome = '"
    + nome + "'";
```

Regra de ouro: sempre PreparedStatement.

PreparedStatement

Usa `?` e você preenche os valores separados — seguro e mais rápido.

```
// valor entra pelo ?
String sql = "... WHERE nome = ?";
ps.setString(1, nome);
```


INSERT com PreparedStatement.

```
String sql = "INSERT INTO produto "  
    + "(nome, preco, quantidade, categoria) "  
    + "VALUES (?, ?, ?, ?)";  
  
PreparedStatement ps = con.prepareStatement(sql);  
ps.setString(1, "Notebook");  
ps.setDouble(2, 3500.0);  
ps.setInt(3, 15);  
ps.setString(4, "Eletrônicos");  
ps.executeUpdate();
```

Cada `?` é preenchido por um `set...` na ordem. O `executeUpdate()` é usado para gravar (INSERT, UPDATE, DELETE).

ResultSet: linha por linha.

Quando você faz um SELECT, o banco devolve um `ResultSet` — uma tabela que você percorre com `rs.next()`, lendo cada coluna com `rs.getString(...)`, `rs.getDouble(...)`, etc. É como um cursor que anda de linha em linha.

SELECT e o ResultSet.

```
String sql = "SELECT nome, preco FROM produto";

PreparedStatement ps = con.prepareStatement(sql);
ResultSet rs = ps.executeQuery();

while (rs.next()) {
    String nome  = rs.getString("nome");
    double preco = rs.getDouble("preco");
    System.out.println(nome + " - R$ " + preco);
}
```

O `executeQuery()` devolve o `ResultSet`. O `while` anda linha por linha até acabar.

Feche o que você abriu.

Connection, PreparedStatement e ResultSet precisam ser fechados — senão vazam recursos. O `try-with-resources` fecha tudo sozinho no fim, mesmo se der erro. E o erro do banco vem como `SQLException`, que a gente já sabe tratar.

```
String sql = "SELECT * FROM produto";

try (Connection con = Conexao.abrir();
     PreparedStatement ps = con.prepareStatement(sql);
     ResultSet rs = ps.executeQuery()) {

    while (rs.next()) {
        System.out.println(rs.getString("nome"));
    }
}
catch (SQLException e) {
    System.out.println("Erro no banco: "
        + e.getMessage());
}
```

A GRANDE IDEIA

Cada classe com um papel só.

SQL fica no DAO. O resto do sistema nem sabe que existe
um banco.

O padrão DAO.

DAO = Data Access Object. Uma classe cuja única função é conversar com o banco para uma entidade. O menu e as regras de negócio não escrevem SQL — só chamam `dao.inserir(p)` ou `dao.listar()`. Trocou de banco? Muda só o DAO.

```
// ponto único de conexão, reutilizável
public class Conexao {
    public static Connection abrir()
        throws SQLException {
        return DriverManager.getConnection(
            URL, USER, SENHA);
    }
}
```

A cara do ProdutoDAO.

```
public class ProdutoDAO {  
    public void inserir(Produto p) { }  
    public List<Produto> listar() { }  
    public void atualizar(Produto p) { }  
    public void deletar(int id) { }  
}
```

Quatro métodos, um CRUD completo: **C**riar, **R**ecuperar, atualizar e apagar. O resto do sistema só conhece essas assinaturas.

inserir(Produto p).

```
public void inserir(Produto p) {  
    String sql = "INSERT INTO produto "  
        + "(nome, preco, quantidade, categoria) "  
        + "VALUES (?, ?, ?, ?)";  
  
    try (Connection con = Conexao.abrir();  
        PreparedStatement ps =  
            con.prepareStatement(sql)) {  
  
        ps.setString(1, p.getNome());  
        ps.setDouble(2, p.getPreco());  
        ps.setInt(3, p.getQuantidade());  
        ps.setString(4, p.getCategoria());  
        ps.executeUpdate();  
    }  
    catch (SQLException e) {  
        throw new RuntimeException(  
            "Erro ao inserir produto", e);  
    }  
}
```


listar() → List<Produto>.

```
public List<Produto> listar() {  
    List<Produto> lista = new ArrayList<>();  
    String sql = "SELECT * FROM produto";  
  
    try (Connection con = Conexao.abrir();  
        PreparedStatement ps =  
            con.prepareStatement(sql);  
        ResultSet rs = ps.executeQuery()) {  
  
        while (rs.next()) {  
            lista.add(new Produto(  
                rs.getString("nome"),  
                rs.getDouble("preco"),  
                rs.getInt("quantidade"),  
                rs.getString("categoria")));  
        }  
    }  
    catch (SQLException e) {  
        throw new RuntimeException(  
            "Erro ao listar", e);  
    }  
    return lista;  
}
```

atualizar e deletar (por id).

```
// atualizar o preço
String sql = "UPDATE produto SET preco = ? "
            + "WHERE id = ?";
ps.setDouble(1, novoPreco);
ps.setInt(2, id);
ps.executeUpdate();
```

Mesma estrutura `try-with-resources` do `inserir` — muda só o SQL. E nunca esqueça o **WHERE**.

```
// deletar por id
String sql = "DELETE FROM produto WHERE id = ?";
ps.setInt(1, id);
ps.executeUpdate();
```

CHECKLIST

Melhores práticas de queries.

Sempre `PreparedStatement` com `?` — nunca concatenar String.

Feche recursos com `try-with-resources`.

Selecione só as colunas necessárias (evite `SELECT *` em produção).

Sempre `WHERE` em `UPDATE` e `DELETE`.

Centralize a conexão numa classe utilitária.

Trate `SQLException`.

As 3 lições da aula.

1 • Persistência

Dado na memória some; um **banco persiste** mesmo depois de desligar.

2 • Query segura

`PreparedStatement` + `try-with-resources` = segura e sem vazamento.

3 • Camada só

O **DAO** isola o SQL numa camada — o resto do código fica limpo.

